iDATA
Advanced analytics for business

PRESENTA A

BANCO DE DESARROLLO DE AMÉRICA LATINA

ECOCAF

DOCUMENTO TÉCNICO ECOCAF

HISTORIAL DE VERSIONES DEL DOCUMENTO

Versión	Responsable	Modificación	Fecha
V2	iDATA	Creación de documento	04/04/2025

Nombre del cliente	CAF BANCO DE DESARROLLO DE AMERICA LATINA
Nombre del proyecto	ECOCAF
Responsable iDATA	Leonardo David Gonzalez Muñoz (fullstack) Daniela Osorio Trujillo (Project Analyst)
Cargo	fullstack
Nombre del contacto del cliente	Braulio Salazar
Cargo	
Correo electrónico del contacto	Bsalazar@caf.com

Derechos de Autor y Confidencialidad del documento

IDATA S.A.S.
Derechos Reservados

El contenido de este documento y sus anexos se encuentran protegidos por el Derecho de Autor, y éstos son propiedad de IDATA S.A.S. y BANPRO, por lo anterior, se le debe brindar tratamiento de confidencial. El cliente puede usar dicha información solo con el propósito de “apoyar la administración de la solución” sin poder divulgar su contenido a terceras partes ya que contiene ideas, conceptos y metodologías con propiedad de IDATA S.A.S. La clasificación "confidencial" significa que esta información es sólo para uso de las personas a quienes está dirigida. Se requiere de la autorización expresa y escrita de IDATA S.A.S. para realizar copias totales o parciales de este documento. Esta información no puede ser utilizada por el cliente para fines diferentes a los del objetivo del documento realizado por IDATA S.A.S.

1 Contenido

2 Introducción 5

2.1 Objetivo 5

3 Proceso para extraer las variables y obtener los criterios de elegibilidad: Back-End 6

3.1 Paso 1: Subida de documentos al blob storage 6

3.2 Paso 2: Extracción de variables 8

3.3 Paso 3 Análisis de variables 10

3.3.1 Entrada y Preparación de Datos: 10

3.3.2 Creación de Prompts: 10

3.3.3 Interacción con la IA: 11

3.3.4 Procesamiento de las Respuestas: 11

3.3.5 Retorno: 11

3.4 Paso 4: Insertar proyecto: 13

3.4.1 Validación de los Datos: 13

3.4.2 Limpieza de Datos: 13

3.4.3 Conexión a Cosmos DB: 13

3.4.4 Verificación de Existencia del Documento: 14

3.4.5 Inserción o Actualización del Documento: 14

3.4.6 Manejo de Errores: 14

4 Proceso para extraer las variables y obtener los criterios de elegibilidad: Front-End 16

4.1 EcoCAF 16

4.2 Dashboard 17

4.2.1 Filtros 18

4.2.2 Card_Small 19

4.2.3 Graphics 19

4.3 LoadProyect 20

4.3.1 Table_Documents 21

4.4 Revisión 24

4.4.1 Variable_Extraction 25

4.4.2 Revision_Modelo 28

4.5 Historial 32

4.5.1 Filtros_Historial 34

4.5.2 Tabla_Static 34

4.5.3 Tabla_Historial 35

4.6 Historial_Fases 37

2 Introducción

2.1 Objetivo

El objetivo principal de este proyecto es desarrollar e implementar un sistema y centralizar la información relacionada con el impacto verde de CAF mediante el seguimiento de indicadores clave de proyectos verdes financiados con fondos internacionales. Esto permitirá:

- **Optimizar la toma de decisiones estratégicas** en las distintas instancias de CAF (Directorio, CDI, Presidencia y Vicepresidencia), basándose en datos precisos, actualizados y fácilmente accesibles.
- **Facilitar el análisis y evaluación de proyectos verdes** mediante el uso de herramientas digitales que permitan identificar oportunidades de financiamiento, calcular porcentajes de enverdecimiento y determinar combinaciones de fuentes de financiamiento verde.
- **Generar reportes e informes dinámicos** que incluyan indicadores y métricas relevantes, promoviendo una gestión ágil y transparente del progreso hacia la meta de ser un banco verde.
- **Escalar y adaptar la solución** para integrar datos de fuentes internas y externas, visualizarlos en dashboards interactivos, y habilitar notificaciones y alertas que respalden la gestión del negocio.

3 Proceso para extraer las variables y obtener los criterios de elegibilidad: Back-End

3.1 Paso 1: Subida de documentos al blob storage

Este código define una ruta de la API /uploadDocuments que permite a los usuarios subir documentos a través de una solicitud POST. La función recibe múltiples archivos, valida que el tipo de documento coincida con el nombre del archivo. Luego, los archivos procesados se almacenan en un sistema de almacenamiento en la nube (Blob Storage) en bloques, y se confirma su carga. Finalmente, la función responde con información sobre los archivos subidos, incluyendo su nombre, tipo de documento y la ruta en el almacenamiento. El proceso incluye manejo de errores para situaciones como archivos faltantes o fallos en la conversión o carga de los archivos.

```python
@app.route("/uploadDocuments", methods=['POST'])
def process_files():
    print("Procesando")
    if 'files' not in request.files:
        return jsonify({"error": "No file part in the request"}), 400

    files = request.files.getlist('files')
    if not files:
        return jsonify({"error": "No files selected for uploading"}), 400

    tipo_documento = request.form.get('tipoDocumento')
    if not tipo_documento:
        return jsonify({"error": "No document type provided"}), 400

    proyectname = request.form.get('proyecto') or "None"
    uploaded_files_info = []

    with tempfile.TemporaryDirectory() as temp_dir:
        for file in files:
            filename = file.filename
            file_path = os.path.join(temp_dir, filename)
            file.save(file_path)

            # Verificación de coincidencia entre los primeros 3 caracteres de tipo_documento y filename
            if tipo_documento.upper() not in filename.upper():
                return jsonify({
                    "error": f"El archivo '{filename}' no coincide con el tipo de documento '{tipo_documento}'"
                }), 400
```

```python
        try:
            # Subir el archivo (PDF o PDF convertido) a Blob Storage
            pathtoblob = f"{proyectname}/{os.path.basename(file_path)}"
            print("Path: ", pathtoblob)
            blob_client = blob_service.get_blob_client(container=source_container_name, blob=pathtoblob)

            block_size = 4 * 1024 * 1024
            blocks = []
            with open(file_path, "rb") as data:
                block_id = 0
                while True:
                    block = data.read(block_size)
                    if not block:
                        break
                    block_id_str = str(block_id).zfill(6)
                    blob_client.stage_block(block_id_str, block)
                    blocks.append(block_id_str)
                    block_id += 1

            blob_client.commit_block_list(blocks)

            uploaded_files_info.append({
                "message": "File uploaded successfully",
                "path": pathtoblob,
                "filename": filename,
                "tipoDocumento": tipo_documento
            })

        except Exception as e:
            return jsonify({"error": str(e)}), 500

    return jsonify(uploaded_files_info), 200
```

3.2 Paso 2: Extracción de variables

Este código define una ruta de la API /extract_fields que maneja solicitudes POST para extraer información de documentos. Recibe un archivo a través de su ruta (path) y un tipo de documento, ambos proporcionados en el cuerpo de la solicitud en formato JSON. La función llama a la función extractdocument para procesar el archivo y extraer los campos relevantes, según el tipo de documento especificado. Si los campos no pueden ser extraídos, la API responde con un error. Si la extracción es exitosa, devuelve una respuesta JSON que incluye el nombre del archivo, su ruta, el tipo de documento y los campos extraídos. El código maneja errores tanto en la entrada de datos (si el path no se proporciona) como en el procesamiento del documento, y devuelve un mensaje de error en caso de fallos.

```python
@app.route('/extract_fields', methods=['POST'])
def extract_fields():
    try:
        # Obtener el path del archivo desde la solicitud
        data = request.get_json()
        filename = data.get('path')
        tipo_documento = data.get('tipoDocumento')  # Obtiene el tipo de documento

        if not filename:
            return jsonify({'error': 'Path del archivo no proporcionado'}), 400

        # Llamar a la función extractdocument con el path proporcionado
        fields = extractdocument(filename,tipo_documento)

        # Comprobar si los campos fueron extraídos correctamente
        if fields is None:
            return jsonify({'error': 'No se pudieron extraer los campos del documento'}), 500

        # Crear la respuesta en el formato deseado
        response = {
            "Nombre": filename.split('/')[-1],
            "Path": filename,
            "Tipodoc": tipo_documento,
            "Variables": fields
        }

        # Retornar los campos extraídos como respuesta JSON
        return jsonify(response)

    except Exception as e:
        print(f"Ocurrió un error en extract_fields: {e}")
        return jsonify({'error': 'Ocurrió un error en el servidor'}), 500

from flask import Flask, jsonify, request
```

El siguiente código define la función handleExtractVariables, la cual gestiona el proceso de extracción de datos de un conjunto de archivos seleccionados por el usuario. Al ejecutarse, muestra un modal de carga y luego recorre todos los archivos seleccionados para extraer la información correspondiente. Para cada archivo, la función obtiene el path

desde la respuesta de carga (uploadResponse) y luego llama a la función extractFields para extraer los campos del archivo según el tipo de documento. Si la extracción es exitosa, los datos extraídos se agregan a un array extractedData. Si ocurre algún error (por ejemplo, si el path no está disponible o no se puede extraer la información), se muestra un mensaje de alerta y se detiene el proceso.

Al finalizar el procesamiento de todos los archivos, se cierra el modal y, si hubo algún error, el flujo se detiene. Si todo salió bien, se actualiza el estado global de la aplicación mediante el contexto appStateContext con los datos extraídos y otras informaciones como el proyecto y la fase, luego navega a la página de revisión (/Eco-Caf/Revision).

```
const handleExtractVariables = async () => {
  setShowModal(true);
  const extractedData = [];
  let hasError = false;

  for (let i = 0; i < selectedFiles.length; i++) {
    const path = uploadResponse[i]?.path;
    console.log(`Extrayendo datos del archivo: ${selectedFiles[i].name}, Path: ${path}`);

    if (path) {
      try {
        const data = await extractFields(path, docTypes[i]);
        if (data) {
          extractedData.push(data);
        } else {
          throw new Error(`No se pudieron extraer los campos del archivo ${selectedFiles[i].name}`);
        }
      } catch (error) {
        console.error("Error en la extracción:", error);
        alert(`Error al procesar el archivo ${selectedFiles[i].name}: ${error instanceof Error ? error.message : "Error desconocido"}`);
        hasError = true;
      }
    } else {
      console.error(`Path no encontrado para el archivo ${selectedFiles[i].name}`);
      alert(`Error al procesar el archivo ${selectedFiles[i].name}: El path del archivo no está disponible.`);
      hasError = true;
    }
  }
```

```
  setShowModal(false);

  if (hasError) {
    console.warn("Navegación detenida debido a errores.");
    return;
  }

  appStateContext?.dispatch({ type: "SET_RESULTADOEXTRACCION", payload: extractedData });
  appStateContext?.dispatch({ type: "SET_PROYECTO", payload: Proyecto || appStateContext?.state.proyectoProp });
  appStateContext?.dispatch({ type: "SET_FASE", payload: fase });

  navigate("/Eco-Caf/Revision");
};
```

3.3 Paso 3 Análisis de variables

Este código define una ruta de la API /Model que maneja solicitudes POST. Al recibir la solicitud, la función getmodel obtiene los datos en formato JSON del cuerpo de la solicitud mediante request.get_json(). Luego, intenta procesar estos datos llamando a la función getresult(data). Si la función getresult ejecuta correctamente, la API responde con los resultados bajo la clave Variables en formato JSON. Si ocurre algún error durante el proceso (por ejemplo, si getresult falla), la API captura la excepción y devuelve un mensaje de error con un código de estado HTTP 500.

```
@app.route('/Model', methods=['POST'])
def getmodel():
    data = request.get_json()
    try:
        resultado = getresult(data)
        return jsonify({'Variables': resultado})
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

La función ‘getresult’, recibe un proyecto (en formato de lista de diccionarios) y realiza un análisis sobre los indicadores de un proyecto ecológico usando un modelo de IA (presumiblemente GPT-4). Aquí está cómo funciona:

3.3.1 *Entrada y Preparación de Datos:*

El parámetro project es una lista de diccionarios. La función empieza creando un diccionario vacío llamado diccionario_entrada.
Luego, recorre cada diccionario en la lista project y extrae las variables contenidas en la clave "Variables", actualizando el diccionario_entrada con estos valores.

3.3.2 *Creación de Prompts:*

Se definen dos prompts (lista_prompt) que serán utilizados como contexto para la IA. Cada prompt es un texto que solicita a la IA realizar un análisis sobre el proyecto basado en los datos proporcionados. El primer prompt pide un análisis de los indicadores del proyecto, y el segundo evalúa si el proyecto cumple con los criterios de financiamiento

verde.

3.3.3 *Interacción con la IA:*

La función luego itera sobre lista_prompt, y para cada prompt, se crea un conjunto de mensajes (chat_prompt) que se pasa a un modelo de chat (presumiblemente OpenAI GPT-4).
El modelo es invocado con estos mensajes y se ajusta con parámetros como temperature, frequency_penalty, y presence_penalty para controlar el comportamiento del modelo.

3.3.4 *Procesamiento de las Respuestas:*

Después de recibir la respuesta del modelo, la función extrae el contenido (asumiendo que es un JSON), lo limpia y lo convierte en un diccionario. El resultado se guarda en una lista correspondiente (resultados_gef o resultados_criterios), dependiendo de cuál de los dos prompts se haya procesado.

3.3.5 *Retorno:*

Finalmente, la función devuelve el diccionario combinado dict_final, que contiene toda la información relevante del análisis realizado sobre el proyecto.

En resumen, esta función extrae, analiza y organiza la información del proyecto, haciendo uso de un modelo de IA para realizar el análisis de los indicadores y los criterios de financiamiento verde, y devuelve los resultados de este análisis como un diccionario final.

```python
        completion = client.chat.completions.create(
            model="gpt-4o",
            messages=chat_prompt,
            temperature=0.2,
            frequency_penalty=0,
            presence_penalty=0
        )

        print("OpenAI completado")
        answer = completion.choices[0].message.content

        # Guardamos la respuesta en la lista correspondiente
        if i == 0:  # Asumimos que el primer prompt es para resultados GEF
            resultados_gef.append(answer)
            json_string_gef = answer.strip('```json\n')
            # print("RESULT TO PASE: ",json_string_gef)
            result_gef = json.loads(json_string_gef)
            df_gef = pd.DataFrame([result_gef])
        else:  # Los demás prompts se almacenan en resultados_criterios
            resultados_criterios.append(answer)
            json_string_criterios = answer.strip('```json\n')
            print("RESULT TO PASE: ",json_string_criterios)
            result_criterios = json.loads(json_string_criterios)
            df_criterios = pd.DataFrame([result_criterios])
    # Opcionalmente, puedes imprimir o procesar los resultados aquí
    print("Resultados GEF:", len(resultados_gef))
    print("Resultados Criterios:", len(resultados_criterios))
    dict_final = {**result_gef, **result_criterios,**diccionario_entrada}
    dict_final["ELEGIBLE / NO ELEGIBLE"] = dict_final["CLASIFICACIÓN DEL PROYECTO"]
    # for i, dic in enumerate(project):
    #     dict_final.update(dic["Variables"])
    return dict_final
def getresult(project):
    print("Project: ",project)
    diccionario_entrada = {}
    for i, dic in enumerate(project):
        diccionario_entrada.update(dic["Variables"])



    lista_prompt = [
        f"""Basándote en el siguiente documento como contexto y traduciéndolo previamente al español si es necesario {resumen_gef}. Analiza el pro
        Realiza un análisis de los indicadores y subindicadores que aplican para el proyecto, la medida  y él porque aplica esos indicadores.
        La respuesta debe ser en español, clara, concisa y  en formato json, si el campo no existe, dejarás una string vacía como valor, UNICAMENT
        """,
        f"""Basándote en el siguiente documento como contexto {resumen_criterios}. Analiza el proyecto  {project} teniendo en cuenta que el proyec
        La respuesta debe ser en formato JSON, en español, clara y concisa.
        Solamente responde con la información que se te compartió sin incluir información adicional.
        Realiza una clasificación del proyecto indicando si cumple los criterios de financiamiento verde y en que categoría(s) clasifica.
        La respuesta debe ser en español, clara, concisa y  en formato json, si el campo no existe, dejarás una string vacía como valor, UNICAMENT
        """
    ]

    resultados_gef = []
    resultados_criterios = []

    print("LISTA PROMPT:  ", lista_prompt)

    for i, prompt in enumerate(lista_prompt):
        # Formatea el prompt con los datos del proyecto
        # formatted_prompt = prompt.format(project=project, resultados_gef=resultados_gef, resultado_criterios=resultado_criterios)

        chat_prompt = [
            {"role": "system", "content": "Eres un asistente experto en el análisis y aprobación de financiamiento en proyectos ecológicos"},
            {"role": "user", "content": f"{prompt}"}
        ]
```

3.4 Paso 4: Insertar proyecto:

Esta función maneja una solicitud para insertar un proyecto. Llama a una función de inserción, verifica si hubo un error en el proceso, y devuelve un mensaje con el estado de la operación (éxito o error) en formato JSON, acompañado de un código de estado HTTP correspondiente.

```
@app.route('/insertProyect', methods=['POST'])
def insertar_item():
    data = request.get_json()
    resultado = insertar_item_cosmos(data)

    if "Error" in resultado:
        status_code = 400
    else:
        status_code = 200

    return jsonify({'mensaje': resultado}), status_code
```

La función ‘ insertar_item_cosmos’ se encarga de insertar o actualizar un documento en una base de datos de Cosmos DB. Aquí está su funcionamiento:

3.4.1 *Validación de los Datos:*

La función primero verifica si el dato recibido (data) contiene el campo 'id'. Si no lo tiene, retorna un error indicando que el campo 'id' es necesario.

3.4.2 *Limpieza de Datos:*

Llama a la función limpiar_nulls(data), que parece encargarse de eliminar los valores None dentro del objeto data para asegurar que solo se trabajen con valores válidos.

3.4.3 *Conexión a Cosmos DB:*

Se establece una conexión con Cosmos DB utilizando CosmosClient con el endpoint y

la clave configurada.
Se obtiene el cliente de la base de datos y el contenedor dentro de Cosmos DB donde se almacenarán los documentos.

3.4.4 *Verificación de Existencia del Documento:*

Intenta leer un documento existente usando el id proporcionado en data. Si el documento ya existe, se almacena en existing_item.
Si no existe, se captura la excepción CosmosResourceNotFoundError y se crea un nuevo documento con el id proporcionado.

3.4.4.1 Determinación de la Fase Actual:

Se obtiene el valor del campo 'FASE' en los datos recibidos.
Dependiendo del valor de 'FASE', el documento se actualiza con los datos de la fase correspondiente ('originacion' o 'evaluacion'). Si el valor de 'FASE' no es reconocido, se retorna un error.

3.4.5 *Inserción o Actualización del Documento:*

Se utiliza el método upsert_item para insertar o actualizar el documento en el contenedor de Cosmos DB con los datos actualizados.

3.4.6 *Manejo de Errores:*

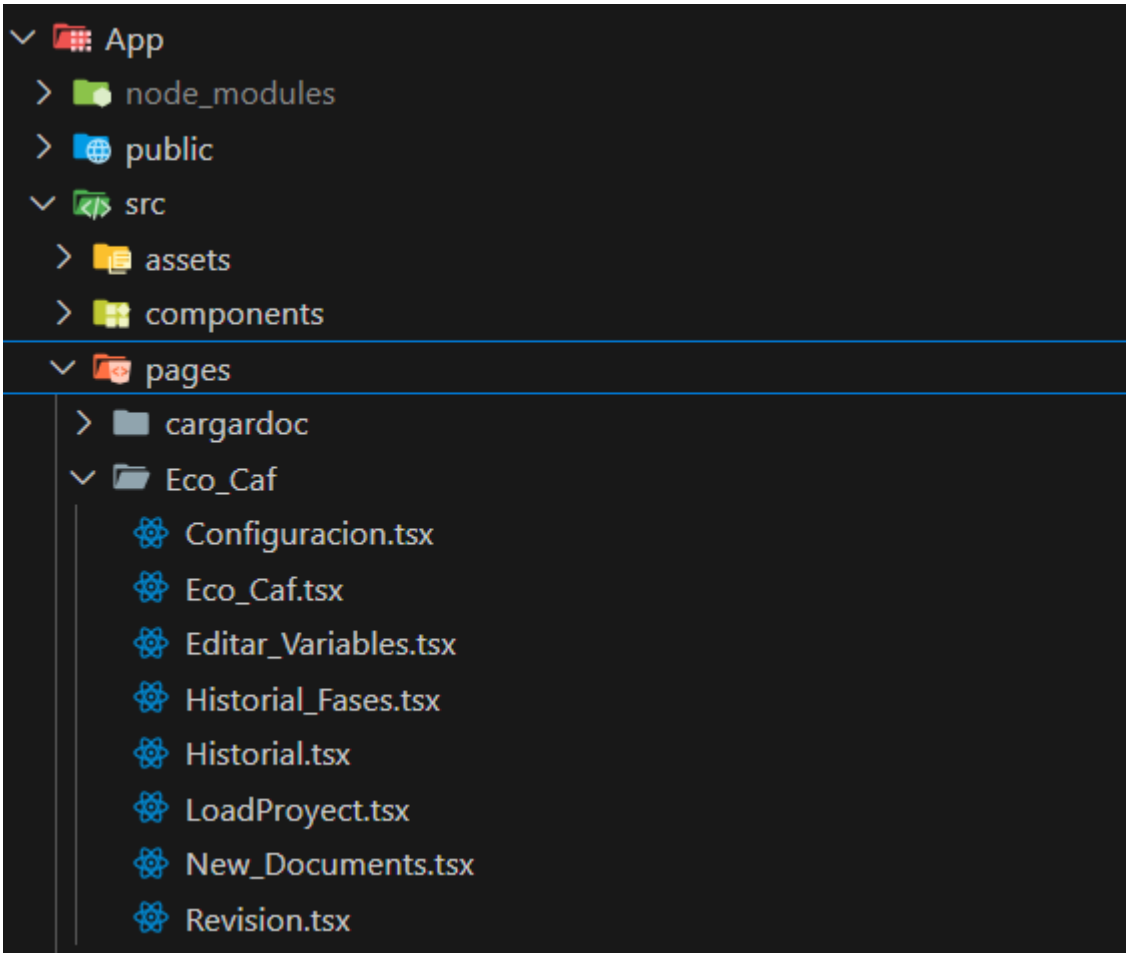
Si ocurre un error relacionado con la conexión a Cosmos DB, se captura la excepción CosmosHttpResponseError y se retorna un mensaje de error detallado.
Si ocurre cualquier otro tipo de error, se captura la excepción general Exception y se retorna un mensaje de error inesperado.
En resumen, esta función valida los datos, limpia los valores nulos, se conecta a Cosmos DB, y luego inserta o actualiza un documento en el contenedor especificado, dependiendo de si el documento ya existe o no. Si la fase del proyecto es válida, los datos se almacenan en el campo correspondiente del documento.

```python
def insertar_item_cosmos(data):
    try:
        # Validar que los datos contengan los campos necesarios
        if 'id' not in data:
            return "Error: El ítem debe tener un campo 'id'"

        # Limpiar los valores None en el data
        data = limpiar_nulls(data)

        client = CosmosClient(endpoint, key)
        database = client.get_database_client(database_name)
        container = database.get_container_client(container_name)

        # Obtener el documento existente (si existe) para mantener las fases anteriores
        existing_item = None
        try:
            existing_item = container.read_item(item=data['id'], partition_key=data['id'])
        except exceptions.CosmosResourceNotFoundError:
            existing_item = {'id': data['id']}  # Crear un nuevo documento si no existe

        # Determinar la fase actual y almacenar los datos en el campo correspondiente
        fase_actual = data.get('FASE', '')

        if fase_actual == "originacion":
            existing_item['originacion'] = data
        elif fase_actual == "evaluacion":
            existing_item['evaluacion'] = data
        else:
            return "Error: Fase desconocida. Debe ser 'originacion', 'evaluacion'."

        # Insertar o actualizar el documento en Cosmos DB
        container.upsert_item(existing_item)
        return 'Item insertado o actualizado exitosamente'
    except exceptions.CosmosHttpResponseError as e:
        return f'Error al conectar: {e.message} - Código de error: {e.status_code}'
    except Exception as e:
        return f'Error inesperado: {str(e)}'
```

4 Proceso para extraer las variables y obtener los criterios de elegibilidad: Front-End

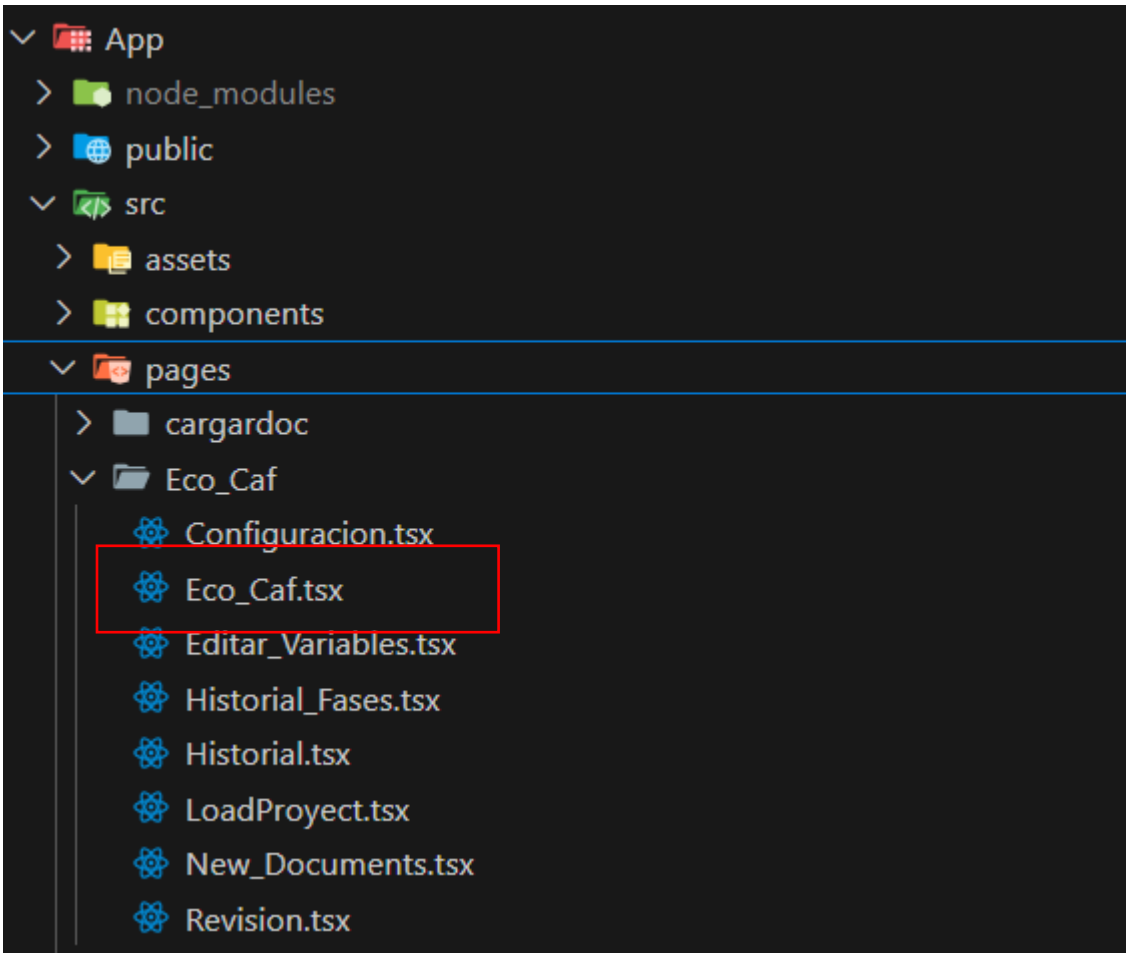
4.1 EcoCAF

La iniciativa ‘EcoCAF’ esta compuesta por 8 paginas de navegacion, cada una de ellas cuenta con sus respectivos componentes, ‘EcoCaf’ se pueden ubicar en la carpta ‘app/src/pages/Eco_Caf’, el cual nos mostrara cada una de las páginas.



4.2 Dashboard

Ubucado en ‘app/src/pages/Eco_Caf/Eco_Caf’, se compone de 3 componentes ‘Filtros, Card_Small, y Graphics’ , las cuales reciben la misma Data, para hacer las funciones del filtrado y mostrar la información correspondiente.



4.2.1 *Filtros*

Componente encargado de hacer la búsqueda detallada de la Data, ya sea por '# operación, fecha, categorías, subcategorías, etc.' Recibe cada uno de los tipos de búsqueda para lograr el objetivo de encontrar la información .

```
<Filtro
  handleProyect={handleProyect}
  setPaisSeleccionado={setPaisSeleccionado}
  paisSeleccionado={paisSeleccionado}
  proyectoSeleccionado={proyectoSeleccionado}
  setProyectoSeleccionado={setProyectoSeleccionado}
  tipoProyecto={tipoProyecto}
  setTipoProyecto={setTipoProyecto}
  operacion={operacion}
  setOperacion={setOperacion}
  criterio={criterio}
  setCriterio={setCriterio}
  fechaInicio={fechaInicio}
  setFechaInicio={setFechaInicio}
  financiamiento={financiamiento}
  setFinanciamiento={setFinanciamiento}
  fechaFin={fechaFin}
  setFechaFin={setFechaFin}
  actividades={actividades}
  setActividades={setActividades}
  nombreProyecto={nombreProyecto}
```

4.2.2 *Card_Small*

Recibe la Data para mostrar la información en tarjetas, tales como, ‘información de montos, porcentajes, incrementos, aceptación de proyectos’

```
<Grid item xs={12}  >
  <Card_Small  kpis={kpis} />
</Grid>
```

4.2.3 *Graphics*

Este componente nos muestra con graficas de barra como circulares, la información de la cantidad de proyectos por país, y los criterios de elegibilidad, como lo son ‘Categoría principal, categoría, subcategoría, indicadores, subindicadores, fuentes’

```
<Grid item xs={12}  >
  <Graphics  kpis={kpis}/>
</Grid>
```

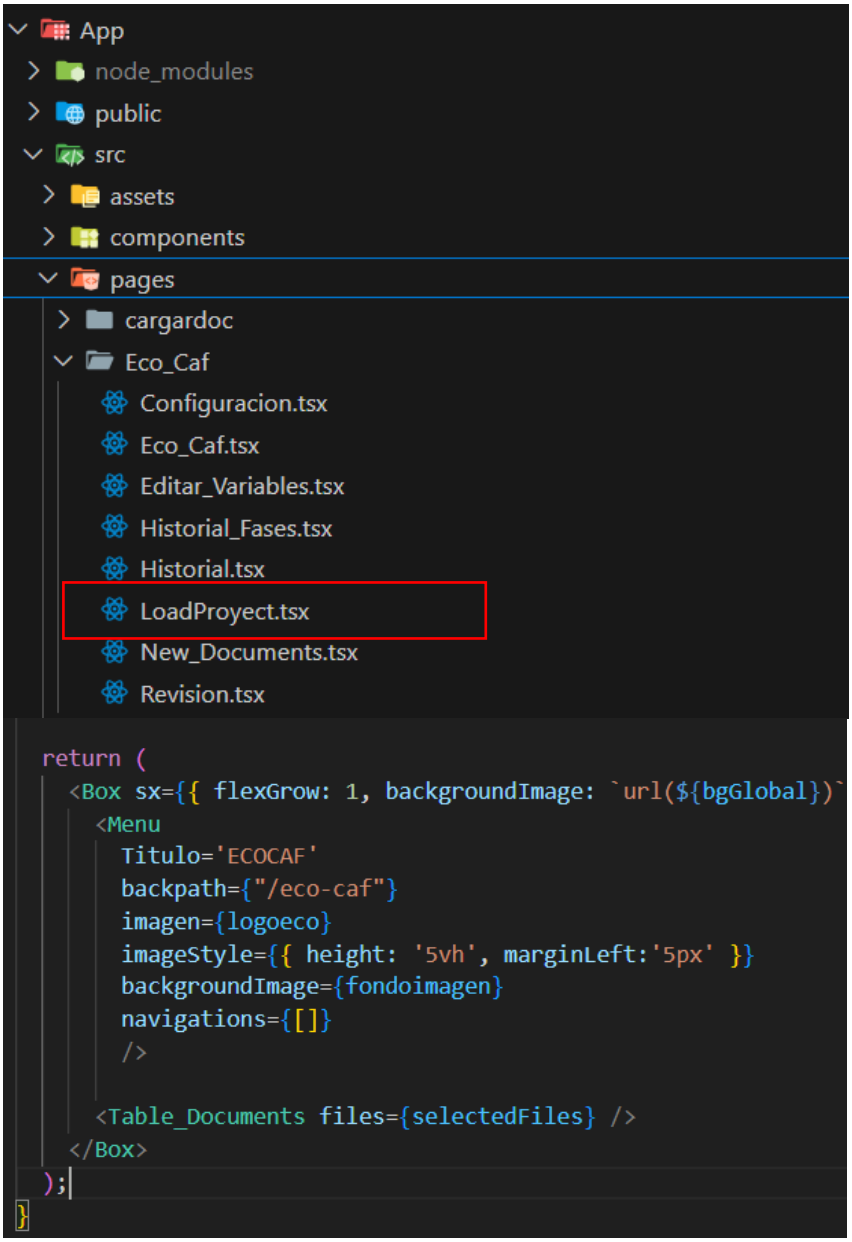
Todo esto funciona con la llamada del endpoint ‘kpis’, el cual nos trae la información de los ‘Kpi’s’ solicitados y con los parámetros necesarios para hacer la búsqueda detallada de cada unos de los proyectos.

```python
@app.route('/kpis', methods=['GET'])
def get_kpis():
    pais = request.args.get('pais', None)
    proyecto = request.args.get('proyecto', None)
    tipoProyecto = request.args.get('tipoProyecto', None)
    fechaInicio = request.args.get('fechaInicio', None)
    fechaFin = request.args.get('fechaFin', None)
    criterio = request.args.get('criterio', None)
    operacion = request.args.get('operacion', None)
    financiado = request.args.get('financiado', None)
    actividad = request.args.get('actividades', None)
    nombreProyecto= request.args.get('nombreProyecto', None)
    categoriaP= request.args.get('categoriaP', None)
    categoria= request.args.get('categoria', None)
    subCategoria= request.args.get('subCategoria', None)
    fase= request.args.get('fase', None)
    indicadores= request.args.get('indicadores', None)
    subIndicadores= request.args.get('subIndicadores', None)
    criterios= request.args.get('criterios', None)
```

```typescript
const fetchKpis = async (filters: Record<string, string>) => {
  setLoading(true);
  try {
    const queryParams = new URLSearchParams(filters).toString();
    const response = await fetch(`${import.meta.env.VITE_BASE_URL}/kpis?${queryParams}`);
    if (!response.ok) {
      throw new Error('Error al obtener los KPIs');
    }
    const data = await response.json();
    console.log('Datos recibidos del endpoint:', data);
    setKpis(data);
    setLoading(false);
  } catch (error) {
    console.error('Error al obtener los KPIs:', error);
    setLoading(false);
  }
};

const fetchAllKpis = async () => {
  setLoading(true);
  try {
    const response = await fetch(`${import.meta.env.VITE_BASE_URL}/kpis`);
    if (!response.ok) {
      throw new Error('Error al obtener todos los KPIs');
    }
    const data = await response.json();
    console.log('Datos recibidos del endpoint (todos):', data);
    setKpis(data);
    setLoading(false);
  } catch (error) {
    console.error('Error al obtener todos los KPIs:', error);
    setLoading(false);
  }
};
```

4.3 LoadProyect

Página la cual nos permite hacer el carge de nuevos proyectos, está formado por dos componente ‘Menu, Table_Documents’



4.3.1 *Table_Documents*

Es el componente el cual nos muestra, el formulario y el ‘Paso 1’, que realiza la selección del proyecto, el # de operación, la fase, el tipo de documento, cuenta con validaciones, para que se pueda realizar la extracción sin cometer algún error, el aplicativo va guiando al usuario con alert, sobre los documentos, para que elija acorde al proyecto, y la extensión que puede utilizar ‘PDF’, tanto campos

obligatorios, como # de operación, fase y tipo de documento, todo esto lo maneja la función ‘handleUploadDocuments’

```
const handleUploadDocuments = async () => {
  const allowedExtensions = ["pdf"];

  if (!alertShown) {
    alert("Favor de revisar que los documentos coincidan con un mismo proyecto.");
    setAlertShown(true); // Evitar que el alert se vuelva a mostrar
    return; // Detener la ejecución de la función
  }

  // Validar los tipos de archivo antes de continuar
  for (let i = 0; i < selectedFiles.length; i++) {
    const fileExtension = selectedFiles[i].name.split('.').pop()?.toLowerCase();

    if (!fileExtension || !allowedExtensions.includes(fileExtension)) {
      alert(`El archivo ${selectedFiles[i].name} no es válido. Solo se permiten archivos PDF.`);
      return; // Detener la ejecución de la función si hay archivos no válidos
    }
  }

  const validProyectoFormat = /^[A-Za-z]{3}\d+$/;
  if (!validProyectoFormat.test(Proyecto)) {
    setErrorProyecto(true);
    return;
  }

  if (!validateFields()) return;

  if (docTypes.some((type) => type === "")) {
    setOpen('Debes seleccionar el tipo de documento en todos los archivos para realizar la extracción de variables');
    return;
  }
```

```
  setOpen('');
  setIsLoading(true);
  setIsUploadClicked(true);
  let totalEstimatedTime = 0;
  const newEstimatedTimes = [];
  let hasError = false; // Variable para rastrear errores

  for (let i = 0; i < selectedFiles.length; i++) {
    setStatus((prevStatus) => {
      const newStatus = [...prevStatus];
      newStatus[i] = "EN PROCESO";
      return newStatus;
    });

    const formData = new FormData();
    formData.append("files", selectedFiles[i]);
    formData.append("tipoDocumento", docTypes[i]);
    formData.append("proyecto", Proyecto);
    formData.append("fase", fase);

    const uploadSuccess = await uploadDocuments(formData, i);

    if (uploadSuccess) {
      const uploadData = await fetch(import.meta.env.VITE_BASE_URL + "/uploadDocuments", {
        method: "POST",
        body: formData,
      });
```

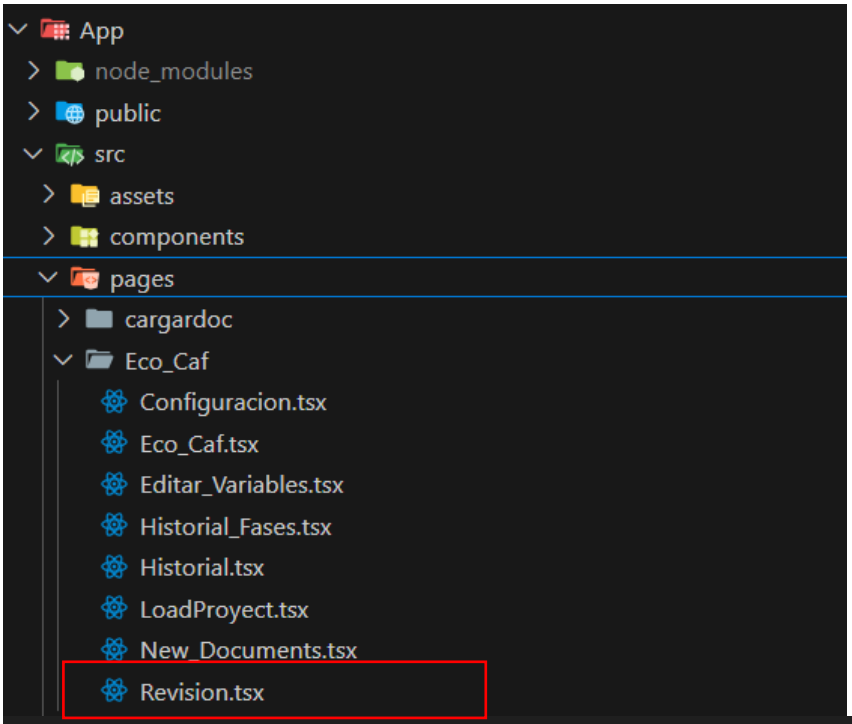
Una vez estando todo correcto con las validaciones, realiza la subida de documentos

Y pasa a extraer las variables con la función ‘handleExtractVariables’, si no hubo errores, toma los resultados, el # de operación (Proyecto) y la fase, haciendo una navegación a ‘/Eco-Caf/Revision’

```
const handleExtractVariables = async () => {
  setShowModal(true);
  const extractedData = [];
  let hasError = false;

  for (let i = 0; i < selectedFiles.length; i++) {
    const path = uploadResponse[i]?.path;
    console.log(`Extrayendo datos del archivo: ${selectedFiles[i].name}, Path: ${path}`);

    if (path) {
      try {
        const data = await extractFields(path, docTypes[i]);
        if (data) {
          extractedData.push(data);
        } else {
          throw new Error(`No se pudieron extraer los campos del archivo ${selectedFiles[i].name}`);
        }
      } catch (error) {
        console.error("Error en la extracción:", error);
        alert(`Error al procesar el archivo ${selectedFiles[i].name}: ${error instanceof Error ? error.message : "Error desconocido"}`);
        hasError = true;
      }
    } else {
      console.error(`Path no encontrado para el archivo ${selectedFiles[i].name}`);
      alert(`Error al procesar el archivo ${selectedFiles[i].name}: El path del archivo no está disponible.`);
      hasError = true;
    }
  }

  if (hasError) {
    console.warn("Navegación detenida debido a errores.");
    return;
  }

  appStateContext?.dispatch({ type: "SET_RESULTADOEXTRACCION", payload: extractedData });
  appStateContext?.dispatch({ type: "SET_PROYECTO", payload: Proyecto || appStateContext?.state.proyectoProp });
  appStateContext?.dispatch({ type: "SET_FASE", payload: fase });

  navigate("/Eco-Caf/Revision");
};
```

4.4 Revisión

Cuenta con 2 componentes, los cuales nos muestran la información que realizo el modelo, como 'Extracción de variables, análisis', cuanta con un aviso al recargar la página, el cual nos avisa, que al recargar la página se perderán los datos




```
useEffect(() => {
  const handleBeforeUnload = (event: BeforeUnloadEvent) => {
    // Agrega mensaje de advertencia para la recarga de página
    event.preventDefault();
    event.returnValue = "Si recargas, se perderán los datos.";

    // Configura una señal en localStorage para detectar recarga confirmada
    localStorage.setItem("redirectOnReload", "true");
  };

  // Agrega el evento beforeunload solo al cargar el componente
  window.addEventListener("beforeunload", handleBeforeUnload);

  // Detecta si se confirmó una recarga previa
  const shouldRedirect = localStorage.getItem("redirectOnReload") === "true";
  if (shouldRedirect) {
    localStorage.removeItem("redirectOnReload");
    navigate("/Eco-Caf/LoadProyect");
  }
```

4.4.1 *Variable_Extraction*

El primer componente es ‘Variable_Extraction’ la cual se encarga de renderizar la información encontrada de la N cantidad de documentos subidos, nos muestra el pdf de cada documento y sus variables, en caso de contar con más de un documento, se mostrará un botón, ‘Ver más’ el cual nos mostrara el pdf de cada uno con su información

```
      <Menu
        Titulo='ECOCAF'
        backpath={"/Eco-Caf/LoadProyect"}
        imagen={logoeco}
        imageStyle={{ height: '5vh', marginLeft:'5px' }}
        backgroundImage={fondoimagen}
        navigations={[]}
        />

      {isModelAccepted ? (
        <Revision_Modelo />
      ) : (
        <Variable_Extraction onAccept={() => setIsModelAccepted(true)} />
      )}
    </Box>
  );
```

```
  {rows2.length > 1 && (
    <TableCell>
      <Button color="primary" sx={{ textDecoration: "underline" }} onClick={() => handleViewMore(index)}>
        VER MÁS
      </Button>
    </TableCell>
  )}

  const handleViewMore = (index: number) => {
    setSelectedDocumentIndex(index);
    console.log("Documento seleccionado:", rows2[index]);
  };
```

El componente nos muestra la información encontrada, utilizando ‘Grid’ de ‘Material MUI’ para mejorar el responsive.

```
<Grid item xs={12} sm={12} md={12} lg={6} key={key}>
  <Input_Blue
    text={texto}
    value={editedVariables[key] !== undefined ? editedVariables[key] : (typeof value === 'object' ? JSON.stringify(value) : value || '')}
    onChange={(e) => handleVariableChange(key, e.target.value)}
  />
</Grid>
```

Nos permite editar la información de cada una de las variables para el cazo en el que el modelo no haya encontrado alguna, o que este mal escrita

```
const saveEditedVariables = async () => {
  setIsLoading(true);
  try {
    const updatedDocs = [...rows2];
    Object.entries(editedVariables).forEach(([key, value]) => {
      updatedDocs[selectedDocumentIndex].Variables[key] = value;
    });
    appStateContext?.dispatch({ type: "SET_RESULTADOEXTRACCION", payload: updatedDocs });
    setEditedVariables({});

    setTimeout(() => {
      setVariablesUpdate(true);
      setIsLoading(false);
    }, 2000);
  } catch (error) {

    setTimeout(() => {
      alert("Error al actualizar");
      setIsLoading(false);
    }, 2000);
  }
};
```

Una vez realizado estos procesos, podemos hacer el análisis de esta información, utilizando la función ‘handleAccept’, y el endpoint ‘model’ (ya comentado su función en la página 11) pasa a realizar el análisis para mostrar los criterios de elegibilidad

```
const handleAccept = async () => {
  setLoading(true);
  const docs = appStateContext?.state.resultadoextraccion;

  try {
    const extractResponse = await fetch(import.meta.env.VITE_BASE_URL + '/Model', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(docs),
    });

    if (extractResponse.ok) {
      const jsonResponse = await extractResponse.json();
      console.log("After", jsonResponse);
      appStateContext?.dispatch({ type: "SET_RESULTADOEXTRACCION", payload: [jsonResponse] });
      onAccept();
    } else {
      console.log("Error en la extracción");
      alert("Ocurrió un error inesperado en el procesamiento del modelo, Por favor intente nuevamente.");
      setLoading(false);
    }
  } catch (error) {
    alert("Ocurrió un error inesperado en el procesamiento del modelo, Por favor intente nuevamente.");
    console.error("Error en la solicitud", error);
  } finally {
    setLoading(false);
  }
};
```

4.4.2 *Revision_Modelo*

Como primera parte nos muestra las variables básicas del proyecto, como los son '# operación, fecha de evaluación, país, cliente, nombre del proyecto', en caso de no encontrar una variable no la se mostrará en la pantalla, (Ya que puede variar el nombre o la variable depende del tipo de documento 'IAS, DEC, IET')

```
<Grid item xs={12} sm={6} >
  <Text_Variables titulo="TIPO DE DOCUMENTO" valor={`${appStateContext?.state.tipoDoc || ""}`} />
</Grid>
<Grid item xs={12} sm={6} >
  <Text_Variables titulo="NOMBRE DEL DOCUMENTO" valor={`${appStateContext?.state.nombre || ""}`} />
</Grid>
<Grid item xs={12} sm={6}>
  <Text_Variables titulo="NOMBRE DEL PROYECTO" valor={`${row.Variables["NOMBRE DEL PROYECTO"] || ""}`} />
</Grid>
<Grid item xs={12} sm={6}>
  <Text_Variables titulo="# OPERACION" valor={`${appStateContext?.state.proyecto}`} />
</Grid>
{row.Variables["FECHA DE EVALUACIÓN"] && (
  <Grid item xs={12} sm={6}>
    <Text_Variables titulo="FECHA DE EVALUACIÓN" valor={`${row.Variables["FECHA DE EVALUACIÓN"]}`} />
  </Grid>
)}
<Grid item xs={12} sm={6}>
  <Text_Variables titulo="PAÍS" valor={`${row.Variables.PAIS || ""}`} />
</Grid>
{row.Variables["ORGANISMO EJECUTOR / CLIENTE"] && (
  <Grid item xs={12} sm={6}>
    <Text_Variables titulo="ORGANISMO EJECUTOR / CLIENTE" valor={`${row.Variables["ORGANISMO EJECUTOR / CLIENTE"] || ""}`} />
  </Grid>
)}
```

Se nos habilita un botón 'EDITAR VARIABLES' el cual nos permite ir a una página donde podemos cambiar las variables en caso de algún error

```
<Button
  onClick={onEdit}
  sx={{ marginLeft: "auto", width: "10rem", height: "2.54rem", backgroundColor: "white",
>
  <p style={{ color: "green", fontWeight: 600 }}> Editar Variables </p>
</Button>
```

```
const onEdit = () => {
  navigate("/Eco-Caf/Editar");
};
```

En la parte inferior se muestra la información que se encontró al hacer el análisis y nos mostrara el nombre de la fase en la que se encuentra este análisis

```
<Box key={index} paddingBlock={5} borderTop={"1px solid #E9E9E9"}>
  <Typography variant="h6" gutterBottom color={"#005187"} fontWeight={700}>
    DATOS PROCESADOS POR EL MODELO FASE: {appStateContext?.state.fase?.toUpperCase()}
  </Typography>
  <Grid container spacing={2}>
    <Grid item xs={6}>
      <Input_Blue
        text="CATEGORÍAS PRINCIPALES DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA"
        value={`${row.Variables["CATEGORÍAS PRINCIPALES DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA"] || ""}`}
        rows={2}
        multiline
      />
    </Grid>

    <Grid item xs={6}>
      <Input_Blue
        text="CATEGORÍAS DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA"
        value={`${row.Variables["CATEGORÍAS DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA"] || ""}`}
        rows={2}
        multiline
      />
    </Grid>
  </Grid>
```

En caso de ya contar con alguna otra fase se nos mostrara un apartado con el mismo estilo, el cual revisa en la base de datos si ya hay alguna fase con ese mismo # de operación

```
{["originacion", "evaluacion", "formalizacion", "desembolso"].map((fase, faseIndex) => {
  // Verifica si la data existe y contiene la fase específica
  if (data && data[fase]) {
    return (
      <Box key={`${faseIndex}`} paddingBlock={5} borderTop={"1px solid #E9E9E9"}>
        <Typography variant="h6" gutterBottom color={"#005187"} fontWeight={700}>
          DATOS PROCESADOS POR EL MODELO FASE: {fase.toUpperCase()}
        </Typography>
        <Grid container spacing={2}>
          <Grid item xs={6}>
            <Input_Blue
              text="CATEGORÍAS PRINCIPALES DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA"
              value={data[fase]?.["CATEGORÍAS PRINCIPALES DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA"] || ""}
              rows={2}
              multiline
            />
          </Grid>
          <Grid item xs={6}>
            <Input_Blue
              text="CATEGORÍAS DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA"
              value={data[fase]?.["CATEGORÍAS DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA"] || ""}
              rows={2}
              multiline
            />
          </Grid>
        </Grid>
```

Usando el siguiente código con el endpoint ‘fases’ el cual requiere el ‘id’ (# de operación) como realizamos la búsqueda por el proyecto para así verificar si ya cuenta con alguna fase, y la busca en la base de datos

```
useEffect(() => {
  const fetchData = async () => {
    try {
      const response = await fetch(`${import.meta.env.VITE_BASE_URL}/fases/${appStateContext?.state.proyecto}`);
      if (!response.ok) {
        throw new Error(`Error: ${response.status} ${response.statusText}`);
      }
      const data = await response.json();
      setData(data);
      console.log("Datos recibidos del endpoint:", data);
    } catch (error) {
      setError(error as Error);
    }
  };

  fetchData();
```

Para realizar el insertado en la base de datos utilizamos la funcion 'handleAccept' La cual de manda la información que encontró el modelo, y el id, como # de operación, fase, tipo de documento y fecha de creación

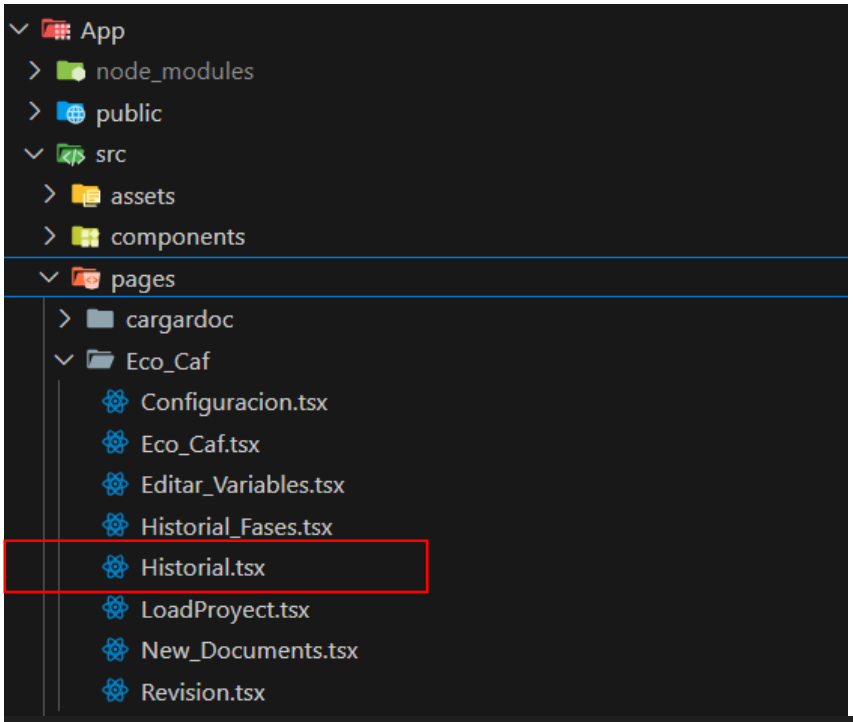
```
const handleAccept = async () => {
  setLoading(true);

  const variables = appStateContext?.state.resultadoextraccion[0]?.Variables;

  try {
    if (variables) {
      variables["id"] = appStateContext?.state.proyecto;
      variables["FASE"] = appStateContext?.state.fase;
      variables["TIPO DOCUMENTO"] = appStateContext?.state.tipoDoc;
      variables["PROYECTO"] = appStateContext?.state.proyecto;
      variables["FECHA CREACION"] = getFormattedDate();

      const extractResponse = await fetch(import.meta.env.VITE_BASE_URL + "/insertProyect", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(variables),
      });

      if (!extractResponse.ok) {
        throw new Error(`Error en la solicitud: ${extractResponse.statusText}`);
      }

      const data = await extractResponse.json();
```

Si se insertó correctamente, nos mostrara un alert de confirmación y su hubo un error, nos dará un alert con el error, al insertar correctamente, nos envía al Dashboard, donde se mostrará la información del nuevo proyecto

```
        console.log("Datos guardados correctamente:", data);
        alert("Datos guardados correctamente.");
      } else {
        throw new Error("No se encontraron variables para el proyecto.");
      }
    } catch (error: any) {
      console.error("Ocurrió un error:", error.message);
      alert(`Error: ${error.message}`);
    } finally {
      setLoading(false);
      // Solo navegar si no hubo errores
      if (!error) navigate("/Eco-Caf");
    }
  };
```

4.5 Historial

Cuenta con 3 componetes ‘Filtros_Historial, Tabla_Static, Tabla_Historial’, aparte de contar con los 3 reportes a descargar los cuales tenemos en los componentes ‘Reporte_Global, Reporte_Id, let’




```
<Filtros_Historial
  proyectoSeleccionado={proyectoSeleccionado}
  setProyectoSeleccionado={setProyectoSeleccionado}
  paisSeleccionado={paisSeleccionado}
  setPaisSeleccionado={setPaisSeleccionado}
  tipoProyecto={tipoProyecto}
  setTipoProyecto={setTipoProyecto}
  <Grid item xs={12} paddingTop={2}>
    <Tabla_Static item={items}/>
    <Tabla_Historial lenghtitems={items.length} generarIn
  </Grid>
```

```
<div ref={reporteRef}>
    {
    showdata &&

    <Reporte_Global kpis={kpis} />

    }

</div>
<div ref={reporteRefID}>
    {
    showdataId &&


    <Reporte_Id id={items[0].id} kpis={kpis}/>

    }

</div>

<div ref={reporteRefIET} >
    {
    showdataIET &&
      <Iet kpis={kpis} id={items[0].id} Terminar={DescargarIET}/>

    }

</div>
```

4.5.1 *Filtros_Historial*

Es un componente parecido a 'Filtros' del Dashboard, realiza la búsqueda, de los proyectos detalladamente con los parámetros, pero utilizando otro endpoint, ya que este solo se encarga de mostrar la información de los proyectos sin 'Kpi's'

```
const fetchItems = async (filters: Record<string, string> = {}) => {
  try {
    setLoading(true);
    const queryParams = new URLSearchParams(filters).toString();
    const response = await fetch(`${import.meta.env.VITE_BASE_URL}/readProyect${queryParams ? `?${queryParams}` : ''}`);

    if (response.ok) {
      const data = await response.json();
      setItems(data.items);
    } else {
      console.error('Error en la respuesta del servidor:', response.statusText);
    }
  } catch (error) {
    console.error('Error al realizar la solicitud:', error);
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchItems();
}, []);
```

4.5.2 *Tabla_Static*

Recibe la data proporcionada por el mismo endpoint 'readProyect' y nos muestra solo cierta información descriptiva el proyecto. Y se muestra a través de una tabla, la cual tiene sus headers, con la información más importante del proyecto y cuenta con un 'Ver más'

```
const headers = [
  'ID',
  'NOMBRE DEL PROYECTO',
  'CLIENTE',
  'MONTO PRÉSTAMO (USD)',
  'MONTO TOTAL DEL PROYECTO (USD)',
  'VICEPRESIDENCIA DE NEGOCIOS',
  'EJECUTIVO DE NEGOCIOS (EJN)',
  'FECHA DE EVALUACIÓN',
  'VER MAS',
]
```

El cual toma el id del proyecto para poder mostrarnos detalladamente toda la información del proyecto seleccionado, utilizando la función 'handleViewDetails', la cual hace la navegación, y manda el id del proyecto

```
<TableCell align="center">
  <Link color="primary" onClick={() => handleViewDetails(row.id)}>
    Ver más
  </Link>
</TableCell>
TableRow>
    const handleViewDetails = (itemId: string) => {
      navigate(`/Eco_caf/Historial/Historial_Fases/${itemId}`);
      appStateContext?.dispatch({ type: "SET_ITEMID", payload: itemId });
  };
```

4.5.3 *Tabla_Historial*

Componente encargado de pasar la información de la tabla para la generación de reportes, contiene la lógica para seleccionar el tipo de reporte deseado, y el formato en el cual se va a descargar el reporte seleccionado

```
          <Typography sx={textos}>TIPO DE INFORME</Typography>
          <Select
            value={tipoInforme}
            onChange={(value:any) => handleTipoInformeChange(value)}
            sx={{
              border: "1px solid #B9E3FF",
              "& .MuiOutlinedInput-notchedOutline": { border: "none" },
              "& .MuiSelect-icon": { color: "#B9E3FF" },
            }}
            size="small"
            fullWidth
          >
            <MenuItem value="">SELECCIONAR</MenuItem>
            <MenuItem value="ReporteId">REPORTE PROYECTO</MenuItem>
            <MenuItem value="Reporte">REPORTE DE RESULTADOS</MenuItem>
            <MenuItem value="Informe">INFORME IET IDO</MenuItem>
          </Select>
        </Grid>
        <Grid item xs={12} md={3}>
          <Typography sx={textos}>FORMATO</Typography>
          <Select
            size="small"
            value={resultado}
            onChange={(value:any) => handleresultado(value)}

            sx={{
              border: "1px solid #B9E3FF",
              "& .MuiOutlinedInput-notchedOutline": { border: "none" },
              "& .MuiSelect-icon": { color: "#B9E3FF" },
            }}
            fullWidth
          >
            <MenuItem value="">SELECCIONAR</MenuItem>
            <MenuItem value="PDF">PDF</MenuItem>
```

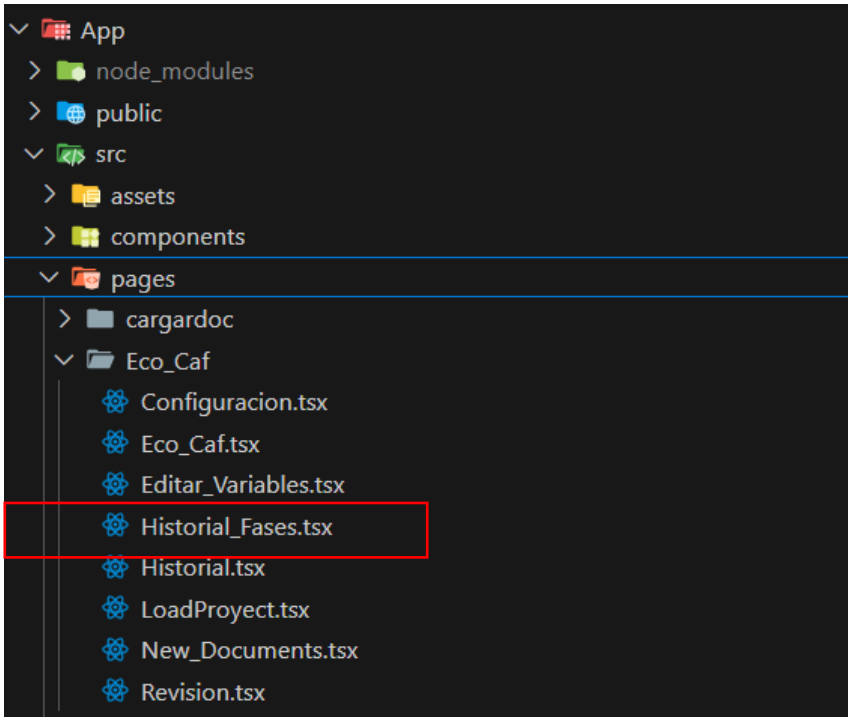
Depende el tipo de reporte seleccionado se ejecuta la función ‘handleGenerarInforme’ el cual recibe 3 funciones, con la lógica para cada tiempo de documento, función que se proporciona desde el componente padre, la cual nos permite realizar la descarga de los 3 reportes , a cada uno de los componentes se les brinda una data, la cual contiene la información general de los proyectos, y la información de un solo proyecto, mostrando graficas tarjetas, tablas, y textos con la información de los proyectos, dando un resumen y características

```
const handleGenerarInforme = () => {
  if(resultado != ""){
    if (tipoInforme === 'Reporte') {
      generarReporteResultados();
    } else if (tipoInforme === 'Informe') {
      if(lenghtitems != 1){
        alert('Para este tipo de reporte, se requiere filtrar por un solo proyecto.');
      }else{
        generarInformeIET_IDO();
      }

    } else if (tipoInforme === 'ReporteId') {
      if(lenghtitems != 1){
        alert('Para este tipo de reporte, se requiere filtrar por un solo proyecto.');
      }else{
        generarReporteResultadosId();
      }
    }
      else {
      console.log('Seleccione un tipo de informe');
    }
  }else{
    alert("Selecciona el tipo de informe y el formato");
  }
};
```

```
const generarReporteResultadosId = () => {
  // Lógica para generar el Reporte de Resultados
  console.log('Generando Reporte de Resultados id');
  generarinfoId();
};

const generarReporteResultados = () => {
  // Lógica para generar el Reporte de Resultados
  console.log('Generando Reporte de Resultados');
  generarInforme();
};

const generarInformeIET_IDO = () => {
  // Lógica para generar el Informe IET_IDO
  console.log('Generando Informe IET_IDO');
  generariet();
};
```

4.6 Historial_Fases

Recibe el id, del proyecto seleccionado y realiza una búsqueda en la base de datos, para obtener la información completa del proyecto



```
useEffect(() => {
    const fetchData = async () => {
        setLoading(true);
        try {
            if (!appStateContext?.state.itemID) {
                throw new Error("No se proporcionó ID del proyecto");
            }

            const response = await fetch(`${import.meta.env.VITE_BASE_URL}/fases/${appStateContext?.state.itemID}`);
            if (!response.ok) {
                throw new Error(`Error: ${response.status} ${response.statusText}`);
            }
            const result = await response.json();
            setData(result);
        } catch (err) {
            setError(err as Error);
        } finally {
            setLoading(false);
        }
    };

    fetchData();
}, [appStateContext?.state.itemID]);
```

Esta data encontrada se pasa al componente ‘Table_Fases’ el cual nos muestra los resultados del modelo, tanto su información básica del proyecto como los criterios de elegibilidad, todo esto lo muestra por cada fase que ya tenga el proyecto

```
<Table_Fases data={data} /
```

```
const [activeTab, setActiveTab] = useState(0);
const navigate = useNavigate();
const appStateContext = useContext(AppStateContext);

const phases = ['originacion', 'evaluacion', 'formalizacion', 'desembolso'];
const currentPhaseData = data && data[phases[activeTab]] ? data[phases[activeTab]] : {};
const phaseMapping: { [key: string]: string } = {
  originacion: 'Originación',
  evaluacion: 'Evaluación',
  formalizacion: 'Formalización',
  desembolso: 'Desembolso',
};
const sinDatos = "SIN DATOS EN FASE"
    <Grid item xs={12}>
      <Tabs
        value={activeTab}
        onChange={handleTabChange}
        sx={{
          justifyContent: 'flex-start',
          display: 'flex',
        }}
      >
        {phases.map((phase, index) => (
          <Tab
            label={`Fase ${index + 1} / ${phaseMapping[phase]}`}
            key={index}
            sx={{
              mr: 2,
              backgroundColor: "#EBEBEB",
              borderRadius: 1,
              '&.Mui-selected': {
                color: '#1a237e',
                borderBottom: '2px solid #1a237e',
                backgroundColor: '#FFFFFF',
              },
            }}
          />
        ))}
      </Tabs>
```

Nos muestra 4 botones los cuales concuerdan las fases en las cuales el proyecto puede pasar, y se nos muestra la información en dos secciones, la básica y los criterios, esto para ver detalladamente cada una de la información por fases

```
return (
  <Grid item xs={12} sm={6} key={key}>
    <Typography variant="subtitle2" sx={textos}>{texto}</Typography>
    {/* Manejo de array o valores simples */}
    {Array.isArray(value) ? (
      value.map((item, index) => (
        <Typography variant="subtitle1" fontWeight={500} key={index}>
          {item}
        </Typography>
      ))
    ) : (
      <Typography variant="subtitle1" fontWeight={500}>
        {value != null ? String(value) : ''}
      </Typography>
    )}
  </Grid>
);
```

```
<Grid item xs={12}>
  <Typography variant="h6" sx={textos}>
    DATOS PROCESADOS POR MODELO  {`FASE ${phaseMapping[phases[activeTab]].toUpperCase()}`}
  </Typography>
  <Box bgcolor={'#FFFFFF'} sx={{ p: 2 }}>
    <Grid container spacing={2}>
      <Grid item xs={12} md={6}>
        <Input_Blue text="CATEGORÍAS PRINCIPALES DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA" value={currentPhaseData["C
      </Grid>
      <Grid item xs={12} md={6}>
        <Input_Blue text="CATEGORÍAS DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA" value={currentPhaseData["CATEGORÍAS DE
      </Grid>
      <Grid item xs={12} sm={6}>
        <Input_Blue text="SUBCATEGORÍAS DE FINANCIAMIENTO VERDE EN LAS QUE CLASIFICA" value={currentPhaseData["SUBCATEGOR
      </Grid>
      <Grid item xs={12}sm={6}>
        <Input_Blue text="ACTIVIDADES ELEGIBLES QUE APLICAN AL PROYECTO" value={currentPhaseData["ACTIVIDADES ELEGIBLES Q
      </Grid>
      <Grid item xs={12}sm={6}>
        <Input_Blue text='¿CUMPLE LOS CRITERIOS DE FINANCIAMIENTO VERDE?' value={currentPhaseData["¿CUMPLE LOS CRITERIOS
      </Grid>
```

Por último, cuenta con un botón el cual nos permite ir nuevamente a cargar más documentos, tomando el id del proyecto actual, para así solo seleccionar la fase que queramos actualizar o si deseamos realizar una nueva fase con el proyecto

```
<Grid item xs={12}>
  <Box display="flex" justifyContent="flex-end">
    <Button
      onClick={hanglenavigate}
      variant="contained"
      sx={{
        backgroundColor: '#4caf50',
        '&:hover': {
          backgroundColor: '#45a049',
        },
      }}
    >
      Cargar más documentos
    </Button>
  </Box>
</Grid>
```

```
const hanglenavigate = () => {
  navigate('/Eco-Caf/NewDocuments');
}
```